jQuery

Unit - 2

Dr. Narayan Joshi

Contents

- The primary features of jQuery
- Setting up a jQuery code environment
- A simple working jQuery script example
- Reasons to choose the jQuery approach over
- plain JavaScript code
- Common JavaScript development tools

Introduction

- Today's World Wide Web is a dynamic environment, and
  - its users set a high bar for both the style and function of sites.
- To build interesting and interactive sites,
  - developers are turning to JavaScript libraries such as jQuery to
  - automate common tasks and to simplify complicated ones.
- One reason the jQuery library is a popular choice is
  - its ability to assist in a wide range of tasks.
- It can seem challenging to know where to begin because
  - jQuery performs so many different functions.
  - Yet, there is a coherence and symmetry to the design of the library;
  - many of its concepts are borrowed from the structure of HTML and CSS.
- In this opening chapter we'll write
  - a functioning jQuery program in just three lines of code

This chapter will cover

- The primary features of jQuery
- Setting up a jQuery code environment
- A simple working jQuery script example
- Reasons to choose the jQuery approach over
- plain JavaScript code
- Common JavaScript development tools

What is jQuery

- jQuery is a
  - fast, small, and feature-rich JavaScript library.
- It simplifies HTML document
  - traversal and manipulation,
  - event handling,
  - animation, and
  - AJAX

  with an easy-to-use API that works across browsers.

What jQuery does

- Access elements in a document:
  - Without a JavaScript library,
    - web developers write many lines of code to
      - traverse the Document Object Model (DOM) tree and
      - locate specific portions of an HTML document's structure.
  - With jQuery,
    - developers have a robust and efficient selector mechanism,
    - making it easy to retrieve the exact piece of the document that
      - needs to be inspected or manipulated.

```
$('div.content').find('p');
```

What jQuery does

- Modify appearance of a web page:
  - CSS offers a powerful method of influencing the way a document is rendered, but
    - it falls short when not all web browsers support the same standards.
  - With jQuery, developers can bridge this gap,
    - relying on the same standards support across all browsers.
  - In addition, jQuery can
    - change the classes or individual style properties applied to
      - a portion of the document even after the page has been rendered.

```
$('ul > li:first').addClass('active');
```

What jQuery does

- **Alter the content of a document**:
  - Not limited to mere cosmetic changes, jQuery can
    - modify the content of a document itself with a few keystrokes.
  - Text can be changed,
  - images can be inserted or swapped,
  - lists can be reordered, or
  - the entire structure of the HTML can be rewritten, and
    - extended—all with a single easy-to-use **API**.
    - `$('#container').append('<a ref="more.html">more</a>');`

What jQuery does

- **Respond to a user's interaction:**
  - Even the most elaborate and powerful behaviors are not useful if
    - we can't control when they take place.
  - The jQuery library offers a way to intercept a wide variety of events, such as
    - a user clicking on a link, without the need to clutter the HTML code itself with event handlers.
    - At the same time, its event-handling API removes browser inconsistencies that often plague web developers.

```
$('button.show-details').click(function() {
    $('div.details').show();
});
```

What jQuery does

- **Animate changes being made to a document**:
  - To effectively implement such interactive behaviors,
  - a designer must also provide visual feedback to the user.
- The jQuery library facilitates this by providing
  - an array of effects such as fades and wipes, as well as
  - a toolkit for crafting new ones.

```
$('div.details').slideDown();
```

What jQuery does

- **Retrieve information from a server without refreshing a page**:
  - This code pattern is known as **Ajax**, which originally stood for **Asynchronous JavaScript and XML**,
  - but has since come to represent a much greater set of technologies for communicating between the client and the server.
- The jQuery library removes the browser-specific complexity from this responsive, feature-rich process, allowing developers to focus on the server-end functionality.

```
$('div.details').load('more.html #content');
```

What jQuery does

- **Simplify common JavaScript tasks:**
  - In addition to all of the document specific features of jQuery,
  - the library provides enhancements to basic JavaScript constructs such as
    - iteration and array manipulation.

```
$.each(obj, function(key, value) {
    total += value;
});
```

Downloading jQuery

- No installation is required. To use jQuery,
  - just need a publicly available copy of the file,
    - either on external site or our own.
  - Since JavaScript is an interpreted language,
    - there is no compilation or build phase to worry about.
  - Whenever we need a page to have jQuery available,
    - we will simply refer to the file's location from a `<script>` element in the HTML document.
- The official jQuery website (http://jquery.com/) always has
  - the most up-to-date stable version of the library, which can be
    - downloaded from the home page.
- This can be replaced with a compressed version in
  - production environments.

Downloading jQuery ..

- As jQuery's popularity has grown,
  - companies have made the file freely available through their Content Delivery Networks (CDNs):
    - Google (https://developers.google.com/speed/libraries/devguide),
    - Microsoft (http://www.asp.net/ajaxlibrary/cdn.ashx), and
    - the jQuery project itself (http://code.jquery.com)

  offer the file on powerful, low-latency servers distributed around the world for fast download (regardless of the user's location).

- While a CDN-hosted copy of jQuery has speed advantages due to
  - server distribution and caching, using
    - a local copy can be convenient during development.
- Throughout our semester, we'll use
  - a copy of the file stored on our own system,
    - which will allow us to run our code whether we're connected to the Internet or not.

jQuery local installation

- Go to the https://jquery.com/download/
  - to download the latest version available.
- Now put downloaded jquery-3.7.1.min.js file in
  - a directory of your website, e.g. /jquery.
- Now you can include jquery library in your HTML file as – **1-invoke-jquery-from-local.html**

```
<html>
   <head>
      <title>The jQuery Example</title>
      <script type = "text/javascript" src="/jquery/jquery-3.7.1.min.js"></script>
      <script type = "text/javascript">
         $(document).ready(function(){
            document.write("Hello, World!");
         });
      </script>
   </head>

   <body>
      <h1>Narayan</h1>
   </body>
</html>
```

Invoking jQuery from CDN

Example: 2-invoke-jquery-from-CDN.html

```
<html>
   <head>
      <title>The jQuery Example</title>
      <script type = "text/javascript"
       src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js">
      </script>

      <script type = "text/javascript">
         $(document).ready(function(){
            document.write("Hello, World!");
         });
      </script>
   </head>
   <body>
      <h1>Narayan.</h1>
   </body>
</html>
```

Calling jQuery library functions

- As almost everything we do when using jQuery reads or manipulates the Document Object Model (DOM),
  - we need to make sure that we start adding events etc. as soon as the DOM is ready.
- If you want an event to work on your page,
  - you should call it inside the $(document).ready() function.
  - Everything inside it will load as soon as
    - the DOM is loaded and before the page contents are loaded.
- To do this, we register a ready event for the document as follows −

```
$(document).ready(function() {
   // do stuff when DOM is ready
});
```

Start using jQuery

- `jQuery()` function
  - When we open web page in a browser and load jQuery library,
    - it adds a global function named `jQuery()`.
      - The global scope means scope of the window,
      - so the global `jQuery()` function can be called like
        - `window.jQuery()` or
        - `jQuery()` anywhere in your web page.
    - `$` is an alias of `jQuery()`,
      - so you can also use `$()` as a short form of `jQuery()`.
    - i.e.
      - `window.jQuery = window.$ = jQuery = $`.
    - It takes two parameters:
      - selector and context.

Start using jQuery

- `jQuery()` function takes two parameters:
  - selector and context.
  - A selector parameter can be
    - CSS style selector expression for matching a set of elements in a document.
      - For example, if you want to do something with all the div elements then
        - use $ function to match all the div elements as..

- Here, you just pass the tag name of the elements you want to find.
- jQuery (aka $) function will return an array of elements specified as a selector.
- We'll see more about selectors in the next section.

Example: Selector Parameter

```
jQuery('div')

//Or

window.jQuery('div')

//Or

$('div')

//Or

window.$('div')
```

Start using jQuery

- `jQuery()` function takes two parameters:
  - selector and context.
  - The second parameter context
    - can be anything,
    - it can be reference of another DOM element or it can be another jQuery selector.
  - The jQuery will start searching for the elements from the element specified in a context.
  - The jQuery selector may perform faster if you provide context parameter.
  - However, context parameter is optional.

Start using jQuery

- Typically, jQuery interacts with the DOM elements and performs various operations on them.
  - So, we need to detect when the DOM (Document Object Model) is loaded fully,
    - so that the jQuery code can start interacting with the DOM without any error.
- For example,
  - we want to write "Hello World!" to div tag in our web page:
    - The first step is to check when the DOM is loaded fully so that we can find the div element and write "Hello World" to it.
    - If we try to write it before DOM loads,
      - jQuery may not find the div element because it might not be constructed at that time
        - (if you write jQuery script in the <head> tag).
- jQuery API includes built-in function `ready()`, which
  - detects whether a particular element is ready (loaded) or not.
  - Here, we need to check whether a document object is loaded or not using ready() function because
    - document loads entire DOM hierarchy.
  - When the document loads successfully, we can be sure that all DOM elements have also loaded successfully.
- In order to check whether a document object is loaded or not,
  - you need to find a document object using jQuery selector function as shown in
    - 3-window-onload-vs-document-ready.html
    - (The document object is a global object in your browser, so you can also write window.document.)

Find `document` object using jQuery selector

Example: Find Document Object using jQuery Selector

```
$(document)

//Or

$(window.document)

//Or

jQuery(document)

//Or

window.jQuery(document)
```

- The above code finds the global document object.
  - Now, you need to find whether it is loaded or not using ready function as:
    `$(document).ready();`
  - `ready()` takes a callback function as a parameter.
    - This callback function will be called once document is ready.

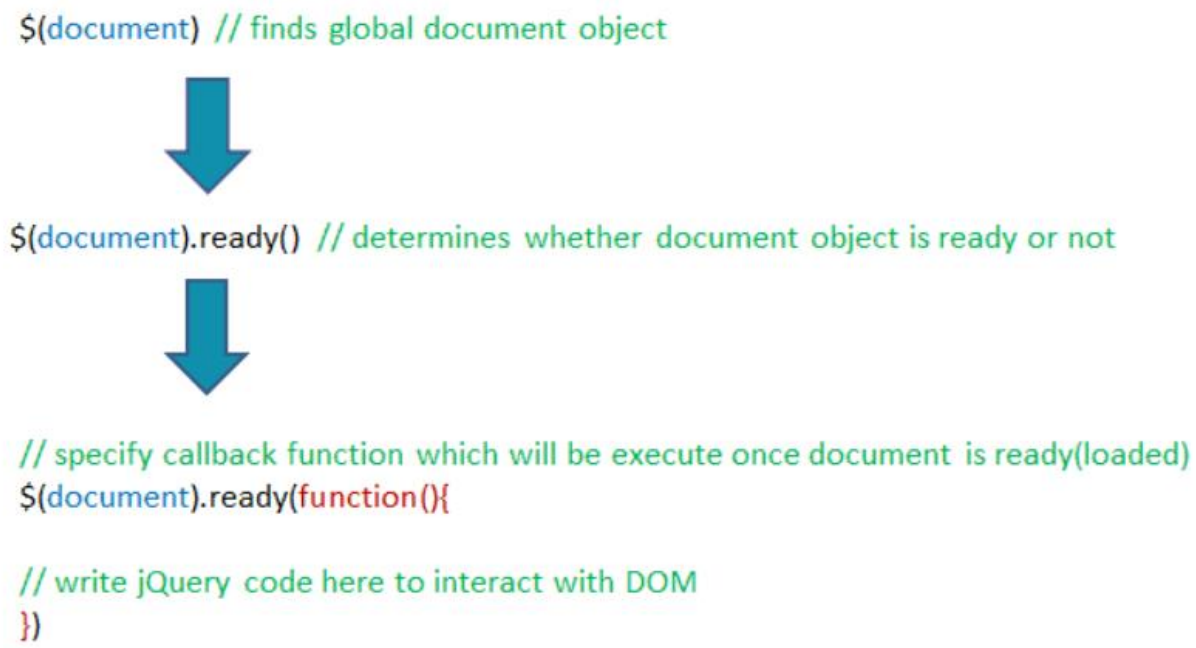
Example: jQuery ready()

```
$(document).ready(function() {

        // DOM is loaded by now...

        // Write jQuery code here...

});
```

Steps to check `document` object is loaded or not

Now, you can start interacting with DOM using jQuery safely in the callback function.

The below figure illustrates the steps:



- This is the first thing you need to write before writing any jQuery code.
  - You always need to check whether a document is ready or not in order to interact with the DOM safely.

window.onload() vs. $(document).ready()

- `$(document).ready()` function
  - determines when the full DOM hierarchy is loaded, whereas
- `window.onload` event is
  - raised when entire window is loaded including DOM, images, css and other required resources.
  - The DOM loads before entire window loads.
- `$(document).ready()`
  - doesn't wait for images to be loaded.
  - It gets called as soon as DOM hierarchy is constructed fully.
- Example:
  - 3-window-onload-vs-document-ready.html

Remember

- jQuery library adds a global function `jQuery()` after jQuery library successfully loaded.
- `$` is a short form of jQuery function. `$() = jQuery() = window.$() = window.jQuery()`
- `$()/jQuery()` is a selector function that selects DOM elements.
- Use jQuery after DOM is loaded fully.
  - Use jQuery `ready()` function to make sure that DOM is loaded fully.
- `window.onload()` & `$(document).ready()` are different.
  - `window.onload()` is fired when the whole html document is loaded with images and other resources, whereas
  - jQuery `ready()` function is fired when DOM hierarchy is loaded without other resources.

jQuery selectors

- How to find DOM element(s) using selectors.
- The jQuery selector
  - enables you to find DOM elements in your web page.
  - Most of the times you will start with
    - selector function `$()` in the jQuery.
- Syntax:

  ```
  $(selector expression, context)
  ```

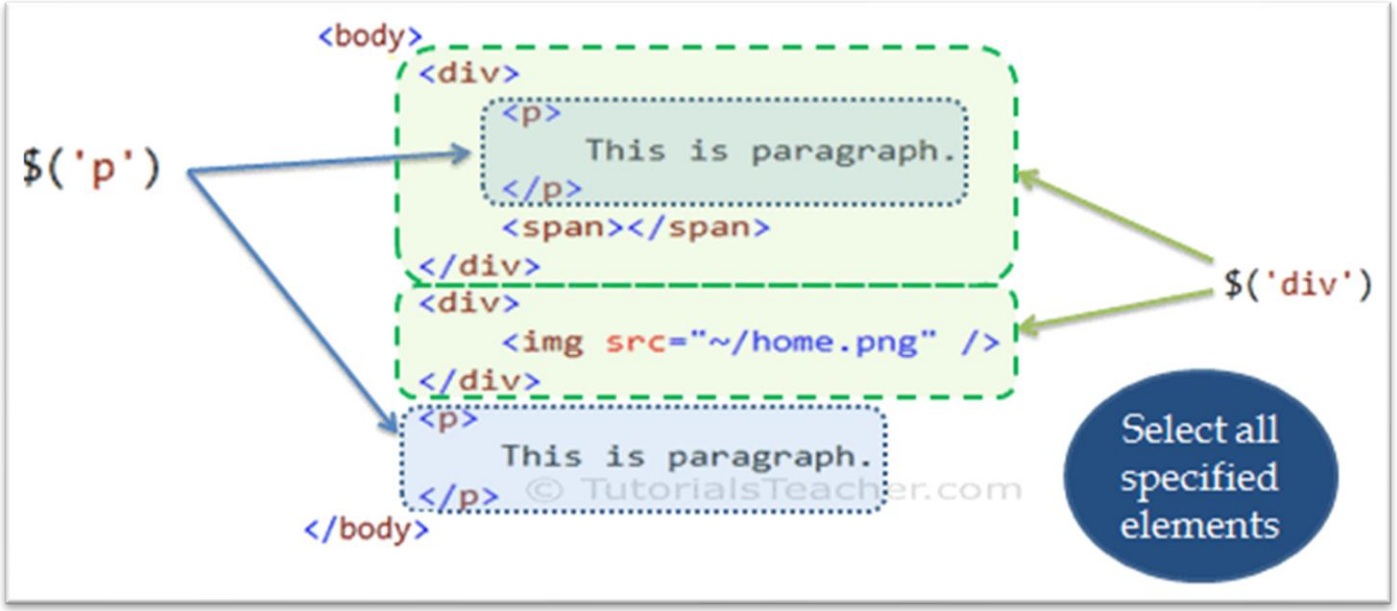
  or

  ```
  jquery(selector expression, context)
  ```

  - The selector expression parameter specifies a pattern to match the elements.
  - The context parameter is optional. It specifies
    - elements in a DOM hierarchy from where jQuery starts searching for matching elements.

jQuery selectors – commonly used selectors

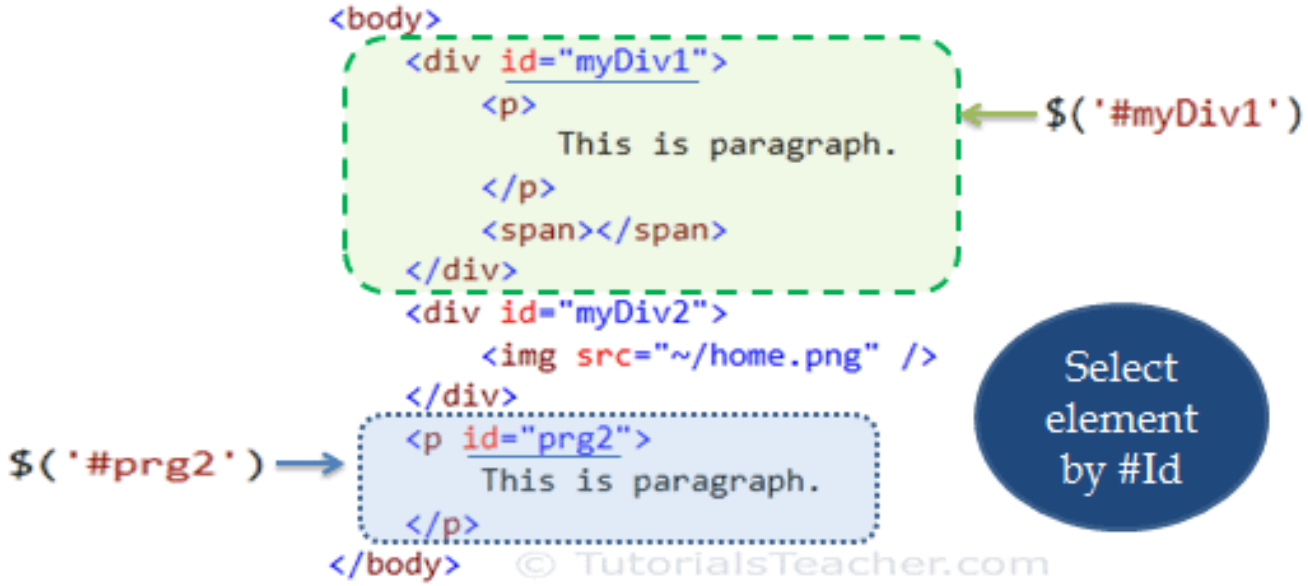
- Select elements by name
  - One of the most common selector pattern
  - Specifying an element name as string e.g.
    - $('p') will return an array of all the <p> elements in a webpage.



- Example: 4-select-element-by-name.html

jQuery selectors – commonly used selectors

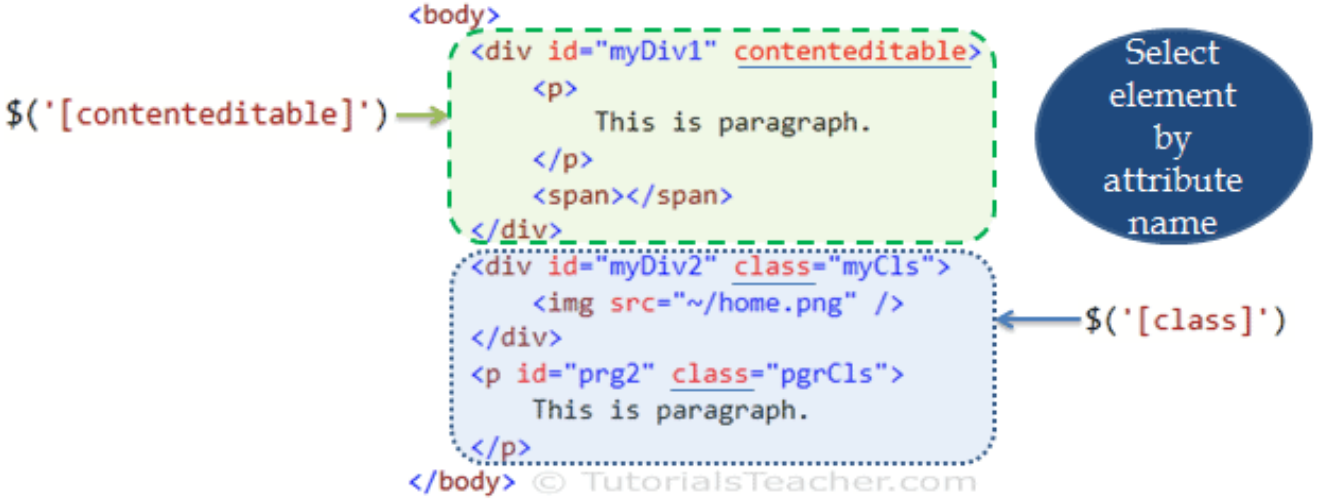
- Select elements by id
  - Get a particular element by using id selector pattern.
    - Specify the id of an element for which you want to get the reference,
      - starting with # symbol.



    - Example: 5-select-element-by-id.html

jQuery selectors – commonly used selectors

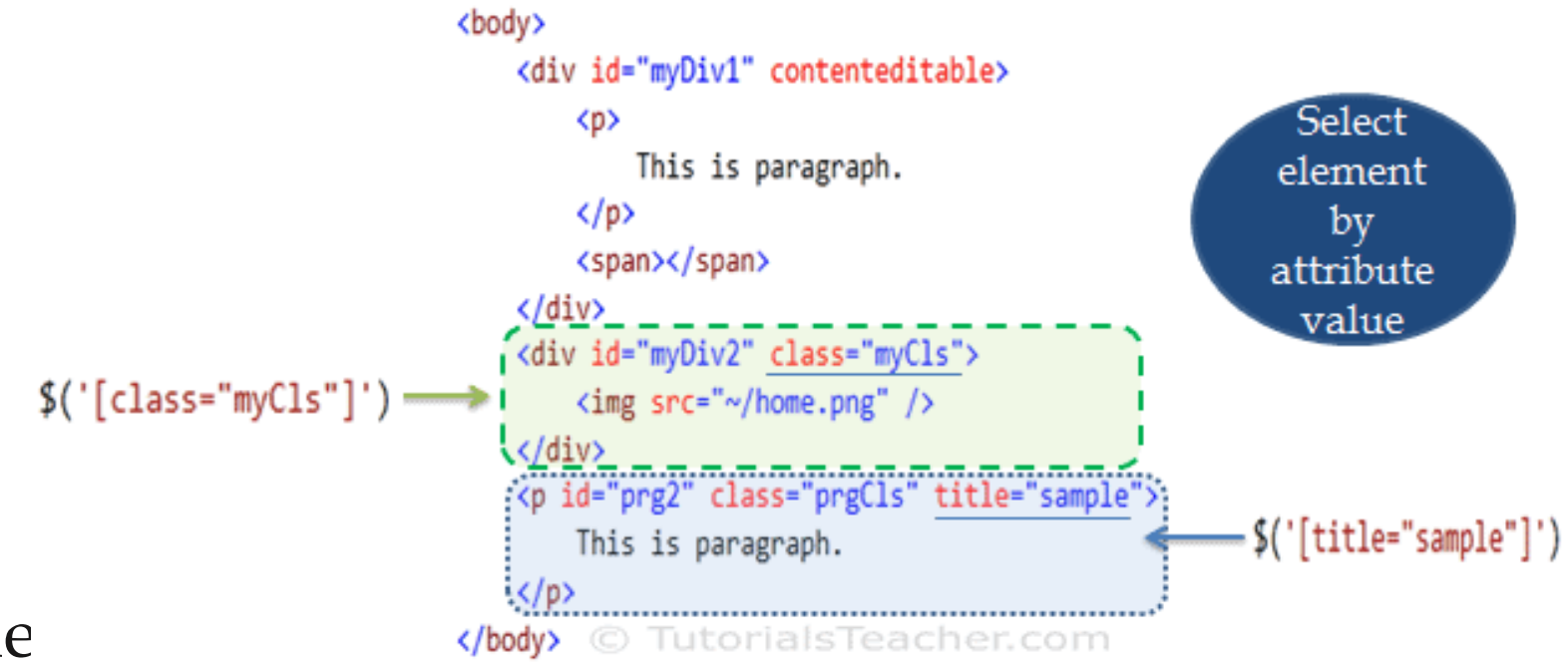
- Select elements by attribute
  - Find an element based on attributes set on it.
    - Specifying an attribute name in square brackets in $ function. E.g.
      - `$('[class]')`
        - will return all the elements that have class attribute irrespective of value.
  - E.g., jQuery returns all the elements that have
    - class or contenteditable attribute irrespective of any value.



- Example: 6-select-element-by-attribute.html

jQuery selectors – commonly used selectors

- Select elements by attribute's specific class value
  - We can also specify a specific value of an attribute in attribute selector.
  - E.g., `$('[class="myCls"]')` will return
    - all the elements which have class attribute with myCls as a value.



- Example
  - 7-select-element-by-attribute-calss-value.html
  - 8-select-element-by-attribute-name.html

jQuery selector patterns

- jQuery provides number of ways to select a specific DOM element(s).
- The following table lists the most important selector patterns.
  - jQuery-selector-patterns.xlsx

jQuery methods

- The jQuery selector
  - finds particular DOM element(s) and wraps them with jQuery object.
  - E.g. document.getElementById() in JavaScript will return DOM object
  - whereas $('#id') will return jQuery object.

```
<html>
<body>
    <div id="myDiv"></div>
</body>
</html>
```

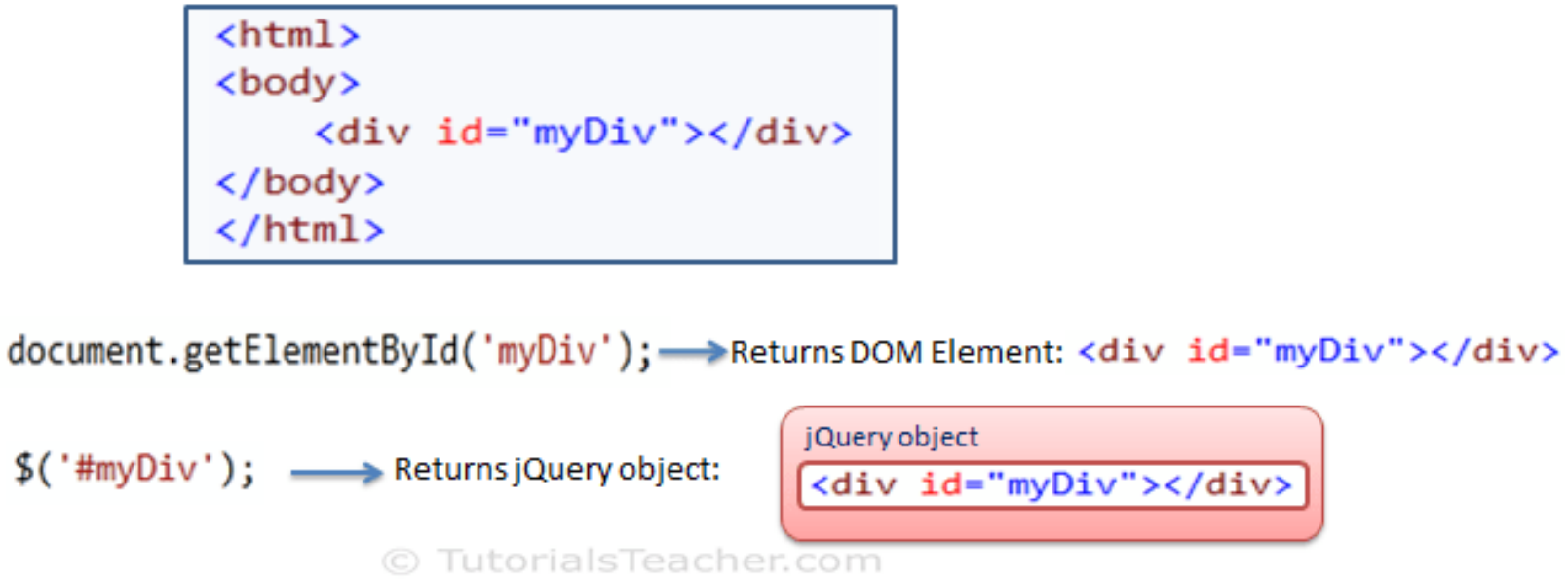
```
document.getElementById('myDiv'); ⟶ Returns DOM Element: <div id="myDiv"></div>

$('#myDiv'); ⟶ Returns jQuery object:
```

jQuery object

```
<div id="myDiv"></div>
```

© TutorialsTeacher.com

jQuery methods



- So, in the above figure,
  - document.getElementById function returns div element
  - whereas jQuery selector returns jQuery object
    - which is a wrapper around div element.
  - So, you can call jQuery methods of jQuery object
    - which is returned by jQuery selector.

jQuery methods

- jQuery provides various methods for different tasks e.g.
  - manipulate DOM,
  - events,
  - ajax etc.
- Different categories of jQuery methods.docx

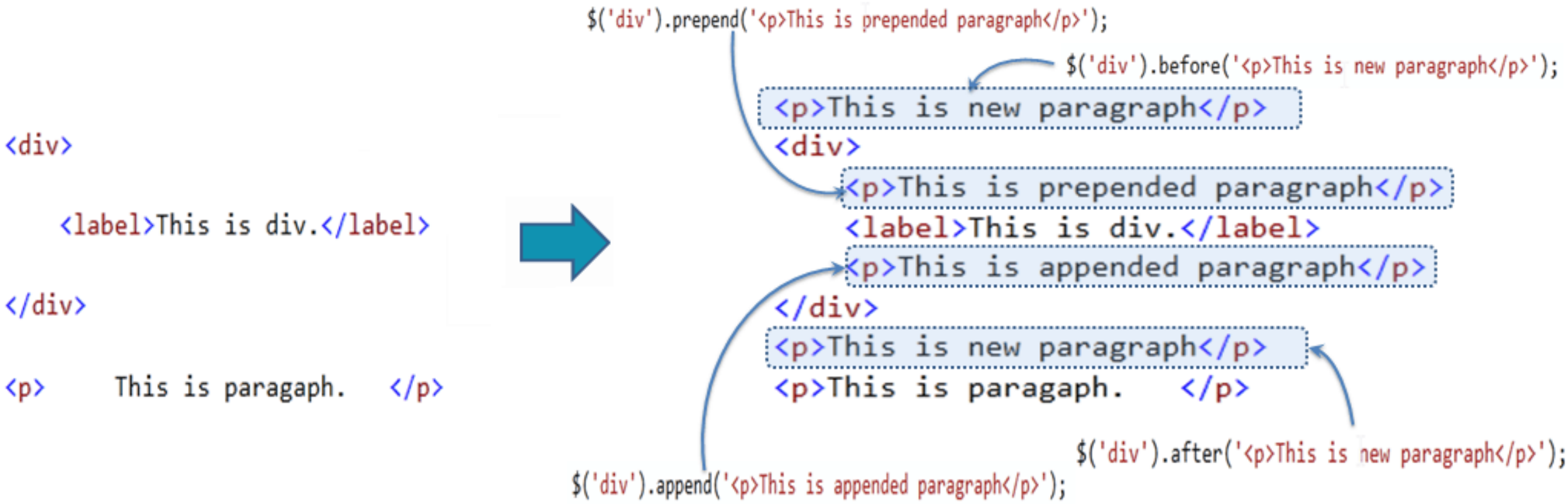
jQuery methods for DOM manipulation

- jQuery provides various methods to
  - add, edit or delete DOM element(s) in the HTML page:

Method	Description
append()	Inserts content to the end of element(s) which is specified by a selector.
before()	Inserts content (new or existing DOM elements) before an element(s) which is specified by a selector.
after()	Inserts content (new or existing DOM elements) after an element(s) which is specified by a selector.
prepend()	Insert content at the beginning of an element(s) specified by a selector.
remove()	Removes element(s) from DOM which is specified by selector.
replaceAll()	Replace target element(s) with specified element.
wrap()	Wrap an HTML structure around each element which is specified by selector.

jQuery DOM manipulation

- Following figure shows
  - how the DOM manipulation methods add new elements.



jQuery before() method – DOM manipulation

- `jQuery after()` method
  - inserts content (new or existing DOM elements) **after** target element(s) which is specified by a selector.
- Syntax:

```
$('selector expression').after('content');
```

  - First of all,
    - specify a selector to get the reference of target element(s) after which
      - you want to add the content and then call after() method.
    - Pass the content string as a parameter.
      - Content string can be any valid HTML element.

- Example: 9-jquery-after-method.html

jQuery after() method – DOM manipulation

- `jQuery before()` method
  - inserts content (new or existing DOM elements) **before** target element(s) which is specified by a selector.
- Syntax:

```
$('selector expression').before('content');
```

- Example: 10-jquery-before-method.html

jQuery append() method – DOM manipulation

- `jQuery append()` method
  - inserts content (new or existing DOM elements) **to the end of** target element(s) which is specified by a selector.
- Syntax:

```
$('selector expression').append('content');
```

- Example: 11-jquery-append-method.html

jQuery prepend() method – DOM manipulation

- `jQuery prepend()` method
  - inserts content (new or existing DOM elements) **at the beginning of** target element(s) which is specified by a selector.
- Syntax:

```
$('selector expression').prepend('content');
```

- Example: 12-jquery-prepend-method.html

jQuery remove() method – DOM manipulation

- `jQuery remove()` method
  - removes element(s) as specified by a selector.
- Syntax:

```
$('selector expression').remove();
```

- Example: 13-jquery-remove-method.html

jQuery replaceAll() method – DOM manipulation

- `jQuery replaceAll()` method
  - replaces all target element(s) with specified element(s).
- Syntax:

```
$('content string').replaceAll('selector expression');
```

- Example: 14-jquery-replaceall-method.html

jQuery wrap() method – DOM manipulation

- `jQuery wrap()` method
  - wrap each target element with specified content element
- Syntax:

```
$('selector expression').wrap('content string');
```

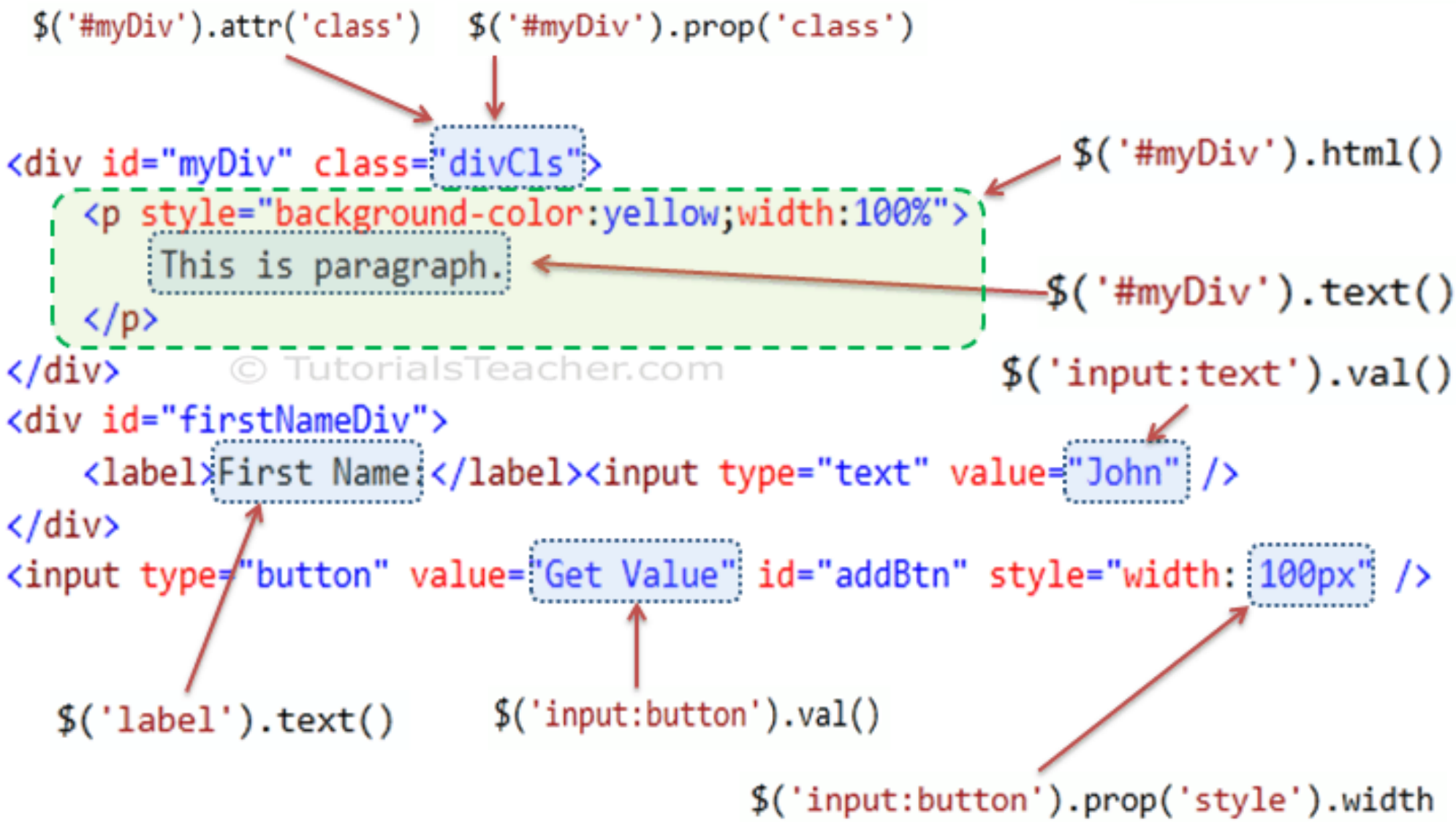
  - Specify a selector to get target elements, and then
    - call wrap method and pass content string to wrap the target element(s).
- Example: 15-jquery-wrap-method.html

jQuery methods for HTML attributes manipulation

- jQuery methods to
  - get or set value of attribute, property, text or html.

jQuery Method	Description
attr()	Get or set the value of specified attribute of the target element(s).
prop()	Get or set the value of specified property of the target element(s).
html()	Get or set html content to the specified target element(s).
text()	Get or set text for the specified target element(s).
val()	Get or set value property of the specified target element.

- jQuery methods to
  - access DOM element's attributes, properties and values.



jQuery attr() method – HTML attribute manipulation

- `jQuery attr()` method
  - to get or set the value of specified attribute of DOM element.
- Syntax:

```
$('selector expression').attr('name','value');
```

  - First of all, specify a selector to get the reference of an element, and
    - call attr() method with attribute name parameter.
    - To set the value of an attribute,
      - pass value parameter along with name parameter.
- Example: 16-jquery-attr-method.html
    - In this example,
      - `$('p').attr('style')` gets style attribute of first `<p>` element in html page. .
      - It does not return style attributes of all <p> elements.

jQuery prop() method – HTML attribute manipulation

- `jQuery prop()` method
  - gets or sets value of specified property to the DOM element(s).
- Syntax:

```
$('selector expression').prop('name','value');
```

- Example: 17-jquery-prop-method.html
  - In this example,
    - In this example,
    - `$('p').prop('style')` returns an object.
    - You can get different style properties by using object.propertyName convention
      - e.g. `style.fontWeight`.
      - Do not include '-' as a property name.

jQuery html() method – HTML attribute manipulation

- `jQuery html()` method
  - gets or sets html content to the specified DOM element(s).
- Syntax:

```
$('selector expression').html('content');
```

  - First of all, specify a selector to get the reference of an element, and
    - call `html()` without passing any parameter to get the inner html content.
    - To set the html content, specify html content string as a parameter..
- Example: 17-jquery-html-method.html
    - In this example,
      - In this example,
      - `$('p').prop('style')` returns an object.
      - You can get different style properties by using object.propertyName convention
        - e.g. `style.fontWeight`.
        - Do not include '-' as a property name.

jQuery text() method – HTML attribute manipulation

- `jQuery text()` method
  - gets or sets text content to the specified DOM element(s).
- Syntax:

```
$('selector expression').text('content');
```

  - First of all, specify a selector to get the reference of an element, and
    - call `text()` without passing any parameter to get the textual content of an element.
    - To set the text content, pass content string as a parameter..
- Example: 18-jquery-text-method.html
- `text()` method only returns text content inside the element and not the innerHtml.

jQuery val() method – HTML attribute manipulation

- `jQuery val()` method
  - gets or sets value property to the specified DOM element(s).
- Syntax:

```
$('selector expression').val('value');
```

- Example: 18-jquery-val-method.html
- `text()` method only returns text content inside the element and not the innerHtml.

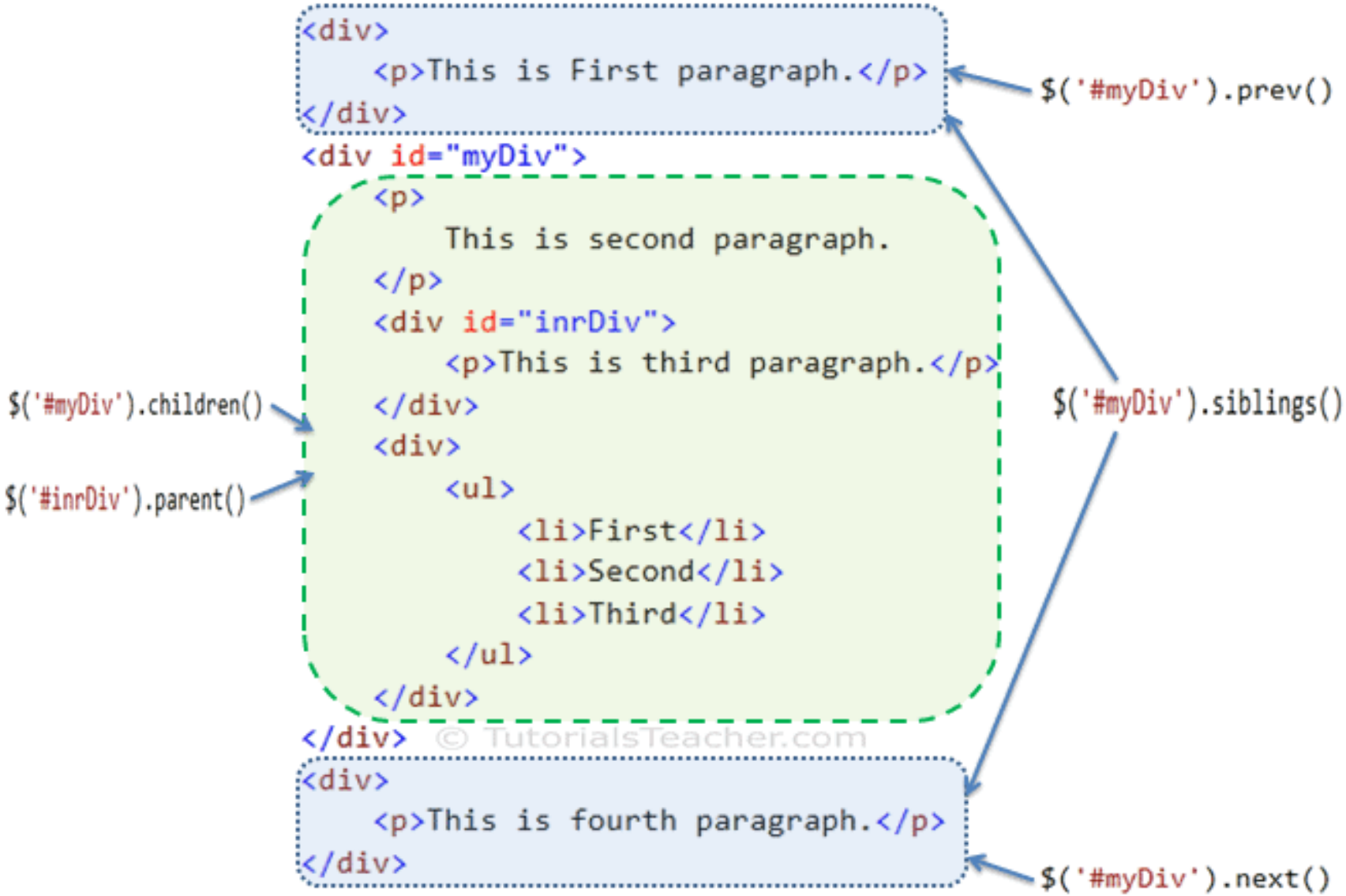
Remember

- jQuery attribute methods are used to
  - manipulate attributes and properties of elements.
- Use the selector to
  - get reference of element(s) and then
    - call jQuery attribute methods to edit it.
- `attr(),prop(),html(),text(),val()`

Various jQuery methods for Traversing DOM elements

jQuery Methods	Description
children()	Get all the child elements of the specified element(s)
each()	Iterate over specified elements and execute specified call back function for each element.
find()	Get all the specified child elements of each specified element(s).
first()	Get the first occurrence of the specified element.
next()	Get the immediately following sibling of the specified element.
parent()	Get the parent of the specified element(s).
prev()	Get the immediately preceding sibling of the specified element.
siblings()	Get the siblings of each specified element(s)

Various jQuery methods for Traversing DOM elements



jQuery each() method – for traversing elements

- jQuery `each()` method
  - iterates over each specified element (specified using selector) and
    - executes callback function for each element.
- Syntax:

```
$('selector expression').each(callback function);
```

  - specify a selector to get the reference of elements, and
    - call each() method with callback function,
      - which will be executed for **each** element.
- Example: 20-jquery-each-method.html
- Notice the '`index`' in above example.

jQuery children() method – for traversing elements

- jQuery `children()` method
  - get the child element of each element specified using selector expression.
- Syntax:

```
$('selector expression').children();
```

  - specify a selector to get the reference of elements, and
    - call children() method to get all the child elements.
- Elements returned from `children()` method can be
  - iterated using `each()` method.
- Example: 21-jquery-children-method.html

jQuery find() method – for traversing elements

- jQuery `find()` method
  - returns all the matching child elements of specified element(s).
- Syntax:

```
$('selector expression').find('selector expression to find child elements');
```

  - Specify a selector to get the reference of an element(s) whose child elements you want to find and then
    - call find() method with selector expression to get all the matching child elements.
    - You can iterate child elements using each method.
- Elements returned from `find()` method can be
  - iterated using `each()` method.
- Example: 22-jquery-find-method.html

jQuery next() method – for traversing elements

- jQuery `next()` method
  - gets the immediately following sibling of the specified element.
- Syntax:

```
$('selector expression').next();
```

  - Specify a selector to get the reference of an element of which
    - you want to get next element and then call `next()` method.
- Elements returned from `next()` method can be
  - iterated using `each()` method.
- Example: 23-jquery-next-method.html

jQuery parent() method – for traversing elements

- jQuery `parent()` method
  - gets the immediate parent element the specified element.
- Syntax:

```
$('selector expression').parent();
```

  - Specify a selector to get the reference of an element of which
    - you want to get next element and then call `next()` method.
- Example: 24-jquery-parent-method.html

jQuery siblings() method – for traversing elements

- jQuery `siglings()` method
  - gets all siblings of the specified element.
- Syntax:

```
$('selector expression').siblings();
```

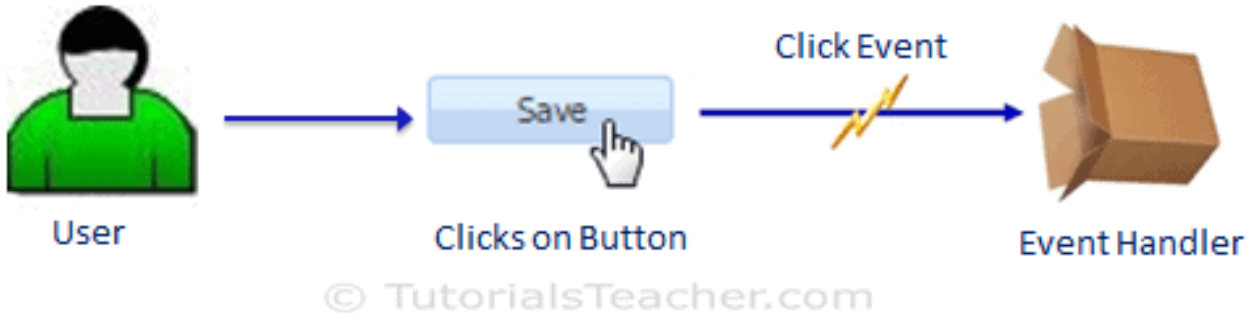
  - Specify a selector to get the reference of an element of which
    - you want to get next element and then call `next()` method.
- Example: 24-jquery-parent-method.html

Remember

- The jQuery traversing methods allow you to
  - iterate DOM elements in a DOM hierarchy.
- Use the selector to get the reference of an element(s), and then
  - call jQuery traversing methods to edit it.
- `each(),children(),find(),first(),parent(), next(),previous(),siblings(),..`

jQuery events

- In most web applications,
  - the user does some action to perform an operation.
  - E.g., user clicks on save button to save the edited data in a web page.
    - Here, clicking on the button is a user's action, which
      - triggers click event and click event handler (function) saves data.



jQuery events

- jQuery event methods
  - The jQuery library provides methods to
    - handle DOM events.
  - Most jQuery methods correspond to native DOM events.
- Query methods and corresponding DOM events:
  - jQuery-event-methods.docx

jQuery events handling

- To handle DOM events using jQuery methods,
  - first get the reference of DOM element(s) using jQuery selector, and
  - invoke appropriate jQuery event method.

- Example: Handle button click event

```
$('#saveButton').click(function() {
   alert('save button clicked.');
};

<input type=button value='save' id='saveButton' />
```

  - First use id selector to get a reference of 'Save' button, and then
    - call click method.
    - Specify handler function as a callback function,
      - which will be called whenever click event of Save button is triggered.

Event object

- jQuery passes an event object to every event handler function.
  - The event object includes
    - important properties and methods for cross-browser consistency
      - e.g. target, pageX, pageY, relatedTarget etc.
  - Example:

```
$('#saveBtn').click(function (eventObj) {
    alert('X =' + eventObj.pageX + ', Y =' + eventObj.pageY);
});

<input type="button" value="Save" id="saveBtn" />
```

  - Example: 26-jquery-event-object.html

`this` keyword in event handler

- `this` keyword in event handler represents a
  - DOM element which raised an event.

  - Example: 27-jquery-this-keyword.html

Mouse hover events

- jQuery provides various methods for
  - mouse hover events e.g.
    - `mouseenter`, `mouseleave`, `mousemove`, `mouseover`, `mouseout` and `mouseup`.
    - Example: 28-jquery-mouse-hover-events.html
  - You can use `hover()` method instead of
    - handling `mouseenter` and `mouseleave` events separately.
  - You can use `hover()` method instead of
    - handling `mouseenter` and `mouseleave` events separately.
    - Example: 28-jquery-mouse-hover-events.html

Event binding using `on()`

- We can
  - attach an event handler for one or more events to the selected elements using `on()` method.
- Internally all of the shorthand methods use `on()` method.
  - `on()` method gives you more flexibility in event binding.
- Syntax:

```
on(types,selector,data,fn);
```

  - `types` = One or more space-separated event types and optional namespaces
  - `selector` = selector string
  - `data` = data to be passed to the handler in event.data when an event is triggered.
  - `fn` = A function to execute when the event is triggered.
- Example: 30-jquery-on-method.html

Event binding using `on()`..

- You can use selector to
  - filter the descendants of the selected elements that trigger the event.

- Example: 31-jquery-on-method.html

Binding multiple events

- Specify multiple event types
  - separated by space.

- Example: 32-jquery-on-method.html

Specify named function as event handler

- You can create separate function and specify it as a handler.
  - This is useful if
    - you want to use the same handler for different events, or
    - events on different elements.

- Example: 33-jquery-named-function-as-event-handler.html

- (jQuery `on()` method is replacement of
  - `live()` and `delegate()` method.)

Event bubbling

- Example: 34-jquery-event-buggling.html
  - As shown in above example,
    - we have handled click event of element in jQuery.
    - So if you click on div,
      - it will display alert message 'DIV clicked’.
    - However, if you click on span,
      - still it will popup alert message SPAN clicked even though we have not handled click event of .
- This is called event bubbling.
- Event bubbles up to
  - the document level in DOM hierarchy till it finds it.

jQuery Event bubbling

```
$('div').click(function (event) {

    alert( event.target.tagName + ' clicked');

});
```